Python Programming
Unit 05 – Lecture 03 Notes
Inheritance and Types of Inheritance

Tofik Ali

February 17, 2026

Contents

1 Lecture Overview

Inheritance is a key OOP feature that allows one class (child) to reuse and extend another class (parent). It reduces duplication and models "is-a" relationships.

2 Core Concepts

2.1 Basic Inheritance

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, sapid):
        super().__init__(name)
        self.sapid = sapid
```

2.2 Types of Inheritance

- **Single:** one parent, one child.
- **Multilevel:** child becomes parent for another class.
- **Hierarchical:** one parent has multiple children.
- **Multiple:** a class has more than one parent (use carefully).

2.3 The Role of `super()`

`super()` calls parent class methods without writing the parent class name. It is especially useful for constructors and for multiple inheritance patterns.

3 Demo Walkthrough

File: `demo/inheritance_demo.py`

The demo shows:

- `Person` base class,
- `Student` subclass,
- `PlacementStudent` as a multilevel subclass.

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: What does `super()` do in a constructor?

Answer: It calls the parent class constructor so parent attributes are initialized correctly.

Checkpoint 2 Solution

Question: Give one real-world example.

Answer (examples): Vehicle → Car, Person → Student, Shape → Circle.

5 Practice Exercises (with Solutions)

Exercise 1: Base + Child Class

Task: Create a base class `Vehicle` (brand) and subclass `Car` (brand, seats).

Solution:

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, seats):
        super().__init__(brand)
        self.seats = seats
```

Exercise 2: Hierarchical Inheritance

Task: Create `Employee` base class and subclasses `Teacher` and `Engineer`.
Solution (idea):

```
class Employee:
    def __init__(self, name):
        self.name = name

class Teacher(Employee):
    pass

class Engineer(Employee):
    pass
```

6 Exit Question (with Solution)

Question: A → (B, C) is which type?
Answer: Hierarchical inheritance.

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: `https://github.com/tali7c/Python-Programming`

Quick Links

Core Concepts **Demo** **Interactive** **Summary**

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Explain inheritance and its purpose
- Create subclasses and reuse parent methods/attributes
- Use `super()` to call parent constructor/methods
- Identify types of inheritance (single, multilevel, multiple, hierarchical)

What is Inheritance?

- Create a new class from an existing class
- Reuse common code and extend behavior
- "is-a" relationship (Student is a Person)

Basic Syntax

```
class Person:
    def __init__(self, name):
        self.name = name

class Student(Person):
    def __init__(self, name, sapid):
        super().__init__(name)
        self.sapid = sapid
```

Types of Inheritance

- Single: A → B
- Multilevel: A → B → C
- Hierarchical: A → (B, C, D)
- Multiple: (A, B) → C

Why `super()`?

- Calls parent class methods safely
- Avoids duplicating parent initialization code
- Important in multiple inheritance (method resolution order)

Demo: Person → Student → PlacementStudent

- File: `demo/inheritance_demo.py`
- Demonstrates:
  - single and multilevel inheritance
  - use of `super()`

Checkpoint 1

Question: What does `super()` do in a subclass constructor?

Checkpoint 2

Question: Give one real-world example of inheritance.

Think-Pair-Share

Discuss:

- Should `Car` inherit from `Engine`? Why or why not?

Key Takeaways

- Inheritance reuses and extends code
- Subclasses can add new attributes and methods
- `super()` helps call parent logic correctly
- Multiple inheritance exists, but use it carefully

Exit Question

What type of inheritance is: A → (B, C)?